SECP3843-01

TUTORIAL AI ALGORITHM

LECTURER : DR ARYATI BINTI BAKRI

NO	NAME	MATRIC NO
1.	PRAVINRAJ A/L SIVABATHI	A23CS0171
2.	DHESHIEGHAN A/L SARAVANA MOORTHY	A23CS0072

a. What is Image Classification

Image classification is a computer vision task where a model is trained to assign a predefined label to an input image based on its visual content. The model learns to identify patterns, shapes, textures, and colours from a labeled training dataset, then applies that knowledge to predict the correct class for images it has never seen before. Real-world applications include medical diagnosis (detecting tumors from X-rays), autonomous vehicles (recognizing road signs), facial recognition, and content moderation. In this tutorial, image classification is applied to the **CIFAR-10** dataset — a benchmark dataset containing 60,000 colour images (32×32 pixels) across 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The dataset is split into 50,000 training images and 10,000 test images.

b. What is CNN?

A **Convolutional Neural Network (CNN)** is a type of deep learning model specifically designed to process grid-structured data like images. Unlike a standard fully connected (dense) network, CNNs preserve the spatial structure of images by applying learnable filters to detect local features hierarchically.

1. Convolutional Layer (Conv2D) Applies filters (kernels) across the image to produce feature maps. Each filter detects specific spatial features such as edges, corners, or textures. In this project, 3×3 filters are used with 32 and 64 filters in the two convolutional layers respectively.

2. Activation Function (ReLU) Applied after each convolution to introduce non-linearity by setting all negative values to zero, allowing the model to learn complex, non-linear patterns.

3. Pooling Layer (MaxPooling2D) Reduces the spatial size of feature maps using a 2×2 window by keeping only the maximum value in each region. This reduces computation, controls overfitting, and makes the model more robust to small shifts or distortions in the image.

4. Flatten Layer Converts the 2D feature maps into a 1D vector so it can be fed into fully connected layers for final classification.

5. Dense Layer Performs high-level reasoning based on the extracted features. The final Dense layer has 10 neurons (one per class) with Softmax activation, converting raw scores into class probabilities.

The CNN model built in this project:

```
cnn = models.Sequential([
    layers.Input(shape=(32, 32, 3)),

    layers.Conv2D(filters=32, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(filters=64, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

c. Evaluation of CNN

Model evaluation measures how well the model generalizes to unseen data. The following metrics are used and their CNN results are shown in the table below.

1. Accuracy The proportion of correctly classified images out of the total test set. Formula: `Accuracy = Correct Predictions / Total Predictions`.

2. Loss (Sparse Categorical Cross-Entropy) Measures how far the predicted probability distribution is from the actual label. Lower loss means better predictions.

3. Precision Of all images predicted as a particular class, how many were actually that class. High precision means few false positives.

4. Recall Of all images that actually belong to a class, how many were correctly identified. High recall means few false negatives.

5. F1-Score The harmonic mean of precision and recall, providing a balanced single metric. Useful when classes are imbalanced.

6. Confusion Matrix A heatmap showing predicted vs. actual classes, used to identify which classes the model confuses most frequently.

CNN Evaluation Metrics Summary:

Metric	CNN Value
Test Accuracy	70.35%
Test Loss	0.9111

d. Steps Hands-On in Python

Step 1 Install Dependencies

```
!python -m pip install --upgrade pip
!pip install tensorflow
```

TensorFlow is the core deep learning framework used to build and train the models.

Step 2 Import Libraries

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import numpy as np
import matplotlib.pyplot as plt
```

TensorFlow and Keras were imported to build and train the neural network models. NumPy was used for numerical operations such as label manipulation and argmax predictions, while Matplotlib was used for data visualization including sample image display and confusion matrix plotting.

Step 3 Load and Preprocess CIFAR-10

```
(X_train, y_train), (X_test, y_test) = datasets.cifar10.load_data()
```

```
X_train = X_train / 255.0
X_test = X_test / 255.0
```

The CIFAR-10 dataset was loaded directly from Keras datasets, which provides 50,000 training images and 10,000 test images. Pixel values were normalized from the original range of [0, 255] to [0, 1] by dividing by 255.0. Normalization ensures the model trains faster and more stably, as large raw pixel values can cause gradient instability during backpropagation.

Step 4 Reshape Labels

```
y_train = y_train.reshape(-1,)
```

```
y_test = y_test.reshape(-1,)
```

The labels loaded from CIFAR-10 have shape (50000, 1), but the sparse_categorical_crossentropy loss function expects 1D arrays of shape (50000,). The reshape(-1,) call flattens the labels into the correct format, ensuring compatibility with the model's loss function during compilation and training.

Step 5 Define Class Names and Visualize Samples

```
classes = ['airplane','automobile','bird','cat','deer','dog','frog','horse','ship','truck']
```

```
def plot_sample(X, y, index):
    plt.figure(figsize=(15,2))
    plt.imshow(X[index])
    plt.xlabel(classes[y[index]])
```

Class labels were defined as a list mapping the integer class indices (0–9) to their human-readable names. The plot_sample() helper function was used to display individual training images with their true class labels. This step verified that the dataset was loaded correctly and that the normalization did not visually distort the images.

Step 6 Build the Baseline ANN Model

```
ann = models.Sequential([
    layers.Flatten(input_shape=(32, 32, 3)),
    layers.Dense(3000, activation='relu'),
    layers.Dense(1000, activation='relu'),
    layers.Dense(10, activation='softmax')
])

ann.compile(
    optimizer='SGD',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

ann.fit(X_train, y_train, epochs=5)
```

The baseline model is an Artificial Neural Network (ANN) with no convolutional layers. The Flatten layer converts the 32×32×3 image into a flat vector of 3,072 values. Two large Dense layers (3,000 and 1,000 neurons) with ReLU activation perform classification. The SGD optimizer was used to update weights. The model was trained for 5 epochs and achieved only approximately 47% test accuracy, highlighting the limitation of fully connected networks for image tasks.

Step 7 Evaluate ANN and Generate Classification Report + Confusion Matrix

```
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np
y_pred = ann.predict(X_test)
y_pred_classes = [np.argmax(element) for element in y_pred]

print('classification report: \n', classification_report(y_test, y_pred_classes))
```

The ANN predictions were generated using predict(), and the predicted class for each image was obtained using argmax() on the softmax output.

Step 8 Build the CNN Model (Enhanced Model)

```
cnn = models.Sequential([
    layers.Input(shape=(32, 32, 3)),

    layers.Conv2D(filters=32, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(filters=64, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

The CNN replaces the flatten-first approach with two Conv2D + MaxPooling blocks that extract spatial features before flattening. The Adam optimizer was chosen over SGD for its adaptive learning rate properties, enabling faster and more stable convergence. The model was trained for 10 epochs, allowing more time for the convolutional filters to learn meaningful feature representations from the CIFAR-10 images.

Step 9 Evaluate CNN

```
cnn.evaluate(X_test,y_test)
```

The CNN was evaluated on the 10,000 test images, achieving a test accuracy of 70.35% and a test loss of 0.9111.

Step 10 Predict and Visualize Results

```
y_pred = cnn.predict(X_test)
y_classes = [np.argmax(element) for element in y_pred]
plot_sample(X_test, y_test, 60)
```

Sample predictions were visualized using plot_sample() to qualitatively confirm the model was classifying images correctly. The steady increase in training accuracy

across 10 epochs (from 47.88% to 78.72%) confirmed the model was learning effectively throughout training.

e. Weaknesses of the Current Model (ANN Baseline)

The baseline model used in this project is an Artificial Neural Network (ANN), which has several limitations when applied to image classification tasks.

1. No Spatial Awareness

The Flatten layer converts the 2D image (32×32×3) into a 1D vector of 3,072 values before any processing takes place. This destroys the spatial relationships between pixels, preventing the model from effectively learning local features such as edges, shapes, and textures that are important for image recognition.

2. Low Accuracy (47%)

The ANN achieved only 47% test accuracy, which indicates limited performance for image classification tasks. This result suggests that the model struggles to learn meaningful image features and distinguish between visually similar classes in the CIFAR-10 dataset.

3. High Parameter Count and Low Efficiency

The ANN uses Dense(3000) and Dense(1000) layers, resulting in a very large number of trainable parameters. This increases memory usage and training time without providing significant improvements in classification accuracy, making the model computationally inefficient for image data.

4. SGD Optimizer Without Tuning

The model uses the Stochastic Gradient Descent (SGD) optimizer without additional techniques such as momentum or learning rate scheduling. As a result, the training process is slower and more sensitive to hyperparameter selection compared to adaptive optimizers such as Adam.

5. Insufficient Training Epochs

The ANN model was trained for only 5 epochs. This relatively small number of training iterations may not be sufficient for the model to converge to an optimal solution, especially when using the SGD optimizer.

6. No Regularization Technique

The model does not include any regularization methods such as Dropout or Batch Normalization. Although the model mainly suffers from underfitting, the absence of regularization techniques may become a limitation when scaling the architecture to larger datasets and more complex tasks.

Overall, the ANN baseline model demonstrates several weaknesses in handling image data, particularly its inability to preserve spatial information. These limitations motivated the use of a Convolutional Neural Network (CNN), which is specifically designed to learn spatial features and improve image classification performance.

f. How to Enhance the Model

The enhanced model addresses the weaknesses of the ANN baseline by replacing it with a Convolutional Neural Network (CNN), which is specifically designed for image classification tasks.

1. Convolutional Layers for Spatial Feature Extraction

The CNN uses Conv2D layers with 3×3 filters to automatically learn important image features such as edges, textures, and shapes. Unlike the ANN, the CNN preserves spatial relationships between pixels, allowing it to extract meaningful visual patterns directly from the input images.

2. MaxPooling for Efficient Downsampling

After each convolutional layer, a MaxPooling2D (2×2) layer is applied to reduce the spatial dimensions of the feature maps. This reduces computational complexity, lowers the number of parameters, and helps the model focus on the most important features while maintaining robustness to small image variations.

3. Progressive Filter Expansion (32 → 64)

The first convolutional layer uses 32 filters to capture low-level features such as edges and corners, while the second convolutional layer uses 64 filters to learn more complex patterns such as shapes and object components. This hierarchical feature learning improves the model's ability to distinguish between different image classes.

4. Adam Optimizer

The CNN uses the Adam optimizer instead of SGD. Adam automatically adjusts the learning rate during training, resulting in faster convergence, more stable learning, and improved overall performance.

5. Increased Training Epochs (10 vs 5)

The CNN model was trained for 10 epochs compared to only 5 epochs for the ANN model. The additional training iterations allow the model to learn more representative features and achieve better classification accuracy.

g. Evaluation: ANN (Baseline) vs CNN (Enhanced)

Metric	ANN (Baseline)	CNN (Enhanced)
Architecture	Flatten → Dense(3000) → Dense(1000) → Dense(10)	Conv2D(32) → Pool → Conv2D(64) → Pool → Dense(64) → Dense(10)
Optimizer	SGD	Adam
Epochs	5	10
Test Accuracy	47%	70.35%
Test Loss	Higher	0.9111
Spatial Feature Learning	No	Yes

ANN Classification Report (per class):

Class	Precision	Recall	F1-Score
airplane	0.55	0.51	0.53
automobile	0.52	0.72	0.61
bird	0.40	0.34	0.37
cat	0.38	0.20	0.26
deer	0.55	0.25	0.34
dog	0.30	0.57	0.39
frog	0.62	0.42	0.50
horse	0.49	0.60	0.54
ship	0.49	0.74	0.59
truck	0.65	0.36	0.47
Overall	**0.50**	**0.47**	**0.46**

The CNN outperforms the ANN by over 23 percentage points in test accuracy. The improvement is consistent across all 10 classes, confirming that convolutional feature extraction is fundamentally better suited for image data than dense layers applied to flattened pixel vectors.

h. Results and Discussion

The experiment clearly demonstrates the superiority of CNNs over ANNs for image classification tasks on CIFAR-10.

The ANN baseline achieved only 47% test accuracy after 5 epochs using the SGD optimizer. The classification report revealed consistently poor F1-scores, with the worst performer being the cat class at 0.26. The fundamental cause is architectural the ANN flattens the input image before any processing, permanently destroying spatial relationships between pixels. Despite having 3,000 and 1,000 neurons in its Dense layers, the model lacks the inductive bias necessary to detect meaningful image features.

The CNN enhanced model achieved 70.35% test accuracy after 10 epochs using Adam. The training accuracy progressed steadily from 47.88% in Epoch 1 to 78.72% by Epoch 10, confirming effective learning throughout training. The gap between training accuracy (78.72%) and test accuracy (70.35%) indicates mild overfitting, which could be addressed with Dropout or data augmentation in future iterations.

The CNN confusion matrix shows improved classification across all 10 classes compared to the ANN, with particular gains on classes that require spatial feature detection. Classes like cat and dog remain the most challenging due to their visual similarity at the low resolution of 32×32 pixels, a limitation inherent to the dataset rather than the model architecture.

Overall, the results validate that CNNs are the appropriate choice for image classification, and that architectural design matters more than raw parameter count for this type of task.

i. Reflection

What we learned: This tutorial provided practical experience with building and comparing ANN and CNN models for image classification using TensorFlow and Keras on the CIFAR-10 dataset. We developed a concrete understanding of why CNNs outperform ANNs for image tasks specifically, the role of convolutional layers in preserving and exploiting spatial structure. We also gained experience interpreting model evaluation metrics including accuracy, loss, classification reports, and confusion matrices.

Challenges faced: One significant challenge was understanding why the ANN performed so poorly despite having very large Dense layers. It initially seemed logical that more neurons would lead to better performance. Through comparison with the CNN, we understood that for image data, *how* features are extracted matters far more than *how many* parameters are used. Another challenge was interpreting the confusion matrix to understand which class pairs were most commonly confused, and connecting those observations back to characteristics of the dataset (e.g., low resolution making cat/dog distinction difficult).

How we solved them: We resolved these challenges by running both models and comparing their outputs side-by-side. Examining the classification report class-by-class helped us identify specific failure points and understand their root causes. Reviewing CNN architecture theory clarified why spatial feature extraction is essential for image classification.

Future improvements: If given more time, we would implement Dropout layers (e.g., 0.3–0.5 drop rate) to reduce overfitting in the CNN, apply Data Augmentation techniques (horizontal flips, random rotations, zoom) to improve generalization, add Batch Normalization between layers for more stable training, increase epochs with a

learning rate scheduler, and explore transfer learning using MobileNetV2 or ResNet50 pre-trained on ImageNet, which can achieve 90%+ accuracy on CIFAR-10 with fine-tuning.

j. Conclusion

This project successfully demonstrated image classification using both ANN and CNN models trained on the CIFAR-10 dataset. The ANN baseline, which used fully connected layers on flattened image vectors, achieved only 47% test accuracy proving insufficient for image classification due to its inability to extract spatial features. The CNN enhanced model, incorporating two Conv2D + MaxPooling blocks with the Adam optimizer, achieved 70.35% test accuracy a significant improvement of over 23 percentage points.

The results confirm that CNNs are inherently better suited to image classification tasks because convolutional layers preserve and exploit the spatial structure of image data, which is the key information source for distinguishing visual categories. Future enhancements such as Dropout, data augmentation, and transfer learning have the potential to push accuracy even further beyond 90%.

Class	Precision	Recall	F1-Score
airplane	0.55	0.51	0.53
automobile	0.52	0.72	0.61
bird	0.40	0.34	0.37
cat	0.38	0.20	0.26
deer	0.55	0.25	0.34
dog	0.30	0.57	0.39
frog	0.62	0.42	0.50
horse	0.49	0.60	0.54
ship	0.49	0.74	0.59
truck	0.65	0.36	0.47
Overall	**0.50**	**0.47**	**0.46**

h. Results and Discussion

The experiment clearly demonstrates the superiority of CNNs over ANNs for image classification tasks.

The ANN baseline achieved only 47% test accuracy after 5 epochs using SGD. The classification report showed that most classes had F1-scores below 0.55, with the cat class performing worst at 0.26. The fundamental reason is that the ANN flattens the image first, destroying all spatial information. With 3,000 neurons in the first dense layer, the model has many parameters but no mechanism to learn meaningful spatial features from pixel grids.

The CNN enhanced model improved test accuracy to 70.35% after 10 epochs with the Adam optimizer. The training accuracy progression showed steady learning from 47.88% in Epoch 1 to 78.72% by Epoch 10. The gap between training (78.72%) and test (70.35%) accuracy indicates some overfitting, which could be addressed with Dropout or data augmentation in future work.

The confusion matrix for the ANN revealed frequent confusion between visually similar classes such as cat/dog and automobile/truck which is expected when spatial features are not properly extracted. The CNN performs significantly better on these challenging class pairs due to its hierarchical feature extraction.

Overall, the results confirm that CNNs are the appropriate architecture for image classification, and that simply adding more parameters to a dense network cannot substitute for the spatial inductive bias that convolutional operations provide.

i. Reflection

What we learned: This tutorial gave us practical hands-on experience with building both an ANN and a CNN for image classification using TensorFlow and Keras. We learned the importance of model architecture choice not just parameter size and how CNNs are specifically suited for image data due to their ability to detect spatial features.

Challenges faced: One major challenge was understanding why the ANN performed so poorly despite having very large Dense layers (3000 and 1000 neurons). It initially seemed like more parameters should mean better performance. Through comparison with the CNN, we understood that for image data, *how* features are extracted matters more than *how many* parameters are used.

Another challenge was interpreting the confusion matrix and classification report to understand *which* classes the model struggled with and *why* for example, cat and dog images are visually similar at 32×32 resolution.

How we solved them: We compared the architecture designs side by side and observed the accuracy gap between models. Running the evaluation metrics helped us identify specific weak points and understand the model's limitations.

Future improvements: If given more time, we would add Dropout layers to reduce overfitting in the CNN, experiment with Data Augmentation to improve generalization, increase the number of epochs and add a learning rate scheduler, and explore pre-trained models like MobileNetV2 via transfer learning, which can achieve 90%+ accuracy on CIFAR-10 with fine-tuning.